LOW LEVEL DESIGN · DRAFT V0.2

Online Judge LLD

The implementation-level companion to the HLD. Every schema, signature, index, comparator edge case and failure mode you need settled before writing code — plus the two places the HLD is wrong.

AUTHOR	STATUS
Meet Modi	Draft — for review
DATE	COMPANION
5 Sep 2026	Online Judge Design Spec
SCOPE	
V0 → V3 implementation	

CONTENTS

00 How to read this
01 Module Map
02 Shared Contracts
03 Data Layer
04 API Service
05 Queue Layer
06 Sandbox Module
07 Judge Worker
08 Event Layer
09 Leaderboard Projector
10 Realtime
11 AI Mentor
12 Ingestor
13 Frontend
14 Configuration
15 Error Envelope and Codes
16 Concurrency and Failure Edge Cases
17 Corrections to the HLD
18 Build Order

00 How to read this

The HLD answers *what the system is and why*. This document answers *how each piece is built*, at the level where two people can implement different modules and have them fit together.

IF YOU WANT	READ
The architecture, capacity model, tradeoffs	HLD §7–§11
Exact schemas, signatures, algorithms, edge cases	this document
What changed after detailed design	§17 — **read this before implementing the sandbox**

Everything here is scoped to V0–V3. Nothing depends on Kubernetes, and every module can be exercised from `docker-compose up`.

01 Module Map

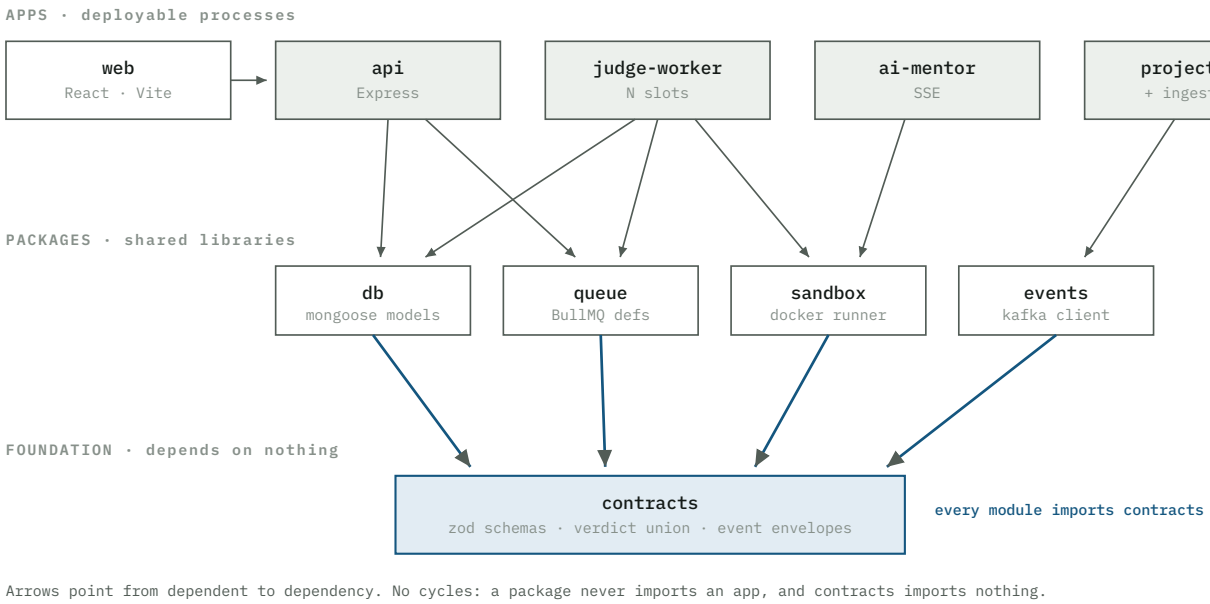


FIG 1 Five deployable processes over four shared packages. `contracts` is the foundation — if two services disagree about what a verdict is, they disagree here first, at compile time.

MODULE	OWNS	MUST NEVER
`api`	HTTP surface, auth, validation, enqueue	Compile or execute user code
`judge-worker`	Judging one submission end to end	Serve HTTP, or trust its own wall clock
`ai-mentor`	Mode routing, context assembly, leak guard	Accept client-supplied context
`leaderboard-projector`	Redis standings projection	Write to `submissions`

MODULE	OWNS	MUST NEVER
`ingestor`	Problem import and normalization	Run on the request path
`contracts`	Types, zod schemas, event envelopes	Import anything

1.1 Repository layout

```
oj/
├── apps/
│   ├── web/                    React + Vite + Monaco
│   ├── api/                    Express: auth, problems, submissions, contests
│   ├── judge-worker/           queue consumer → sandbox → verdict
│   ├── leaderboard-projector/  Kafka consumer → Redis ZSET
│   ├── ai-mentor/              mode router, context builder, leak guard
│   └── ingestor/               source adapters + normalizer
├── packages/
│   ├── contracts/              zod schemas + event types, shared FE/BE
│   ├── db/                     mongoose models + migrations
│   ├── queue/                  BullMQ queue and job definitions
│   ├── sandbox/                docker runner, limits, result parsing
│   └── events/                 kafka producer/consumer wrappers
├── runners/                    one Dockerfile per language
├── infra/                      docker-compose (dev) · terraform + k8s (prod)
└── docs/                       HLD, this document, ADRs, runbooks
```

02 Shared Contracts

One file defines the vocabulary. Everything else imports it, so a rename breaks the build rather than production.

```
// packages/contracts/src/verdict.ts
export const PENDING  = ['QUEUED','COMPILING','RUNNING'] as const;
export const TERMINAL = ['AC','WA','TLE','MLE','RE','CE','OLE','IE'] as const;
export type Pending  = typeof PENDING[number];
export type Terminal = typeof TERMINAL[number];
export type Status   = Pending | Terminal;
export const isTerminal = (s: Status): s is Terminal =>
  (TERMINAL as readonly string[]).includes(s);

// packages/contracts/src/api.ts
export const SubmitBody = z.object({
  problemId: z.string().regex(/^[a-f\d]{24}$/),
  language:  z.enum(['cpp17','c','python3','java17','node20']),
  code:      z.string().min(1).max(64 * 1024),   // 64 KB source cap
  contestId: z.string().regex(/^[a-f\d]{24}$/).optional(),
});
export type SubmitBody = z.infer<typeof SubmitBody>;

// packages/contracts/src/events.ts — every Kafka message wears this envelope
export const Envelope = z.object({
  eventId:   z.string().uuid(),      // producer-generated, used for dedup
  type:      z.string(),             // 'submission.judged'
  version:   z.literal(1),           // bump on breaking payload change
  occurredAt: z.string().datetime(),
  producer:  z.string(),             // 'judge-worker@<hostname>'
  payload:   z.unknown(),
});
```

Rule: a payload field is only ever *added*, never removed or retyped. A breaking change means `version: 2` and a consumer that handles both until the old one drains.

03 Data Layer

3.1 Base collections

Moved here from HLD §10.1, which now carries only a collection-level summary.

```
users {
  _id, username(unique), email(unique), passwordHash, role: 'user'|'setter'|'admin',
  rating, solvedCount, createdAt
}

problems {
  _id, slug(unique), title, statement(md), inputFormat, outputFormat,
  difficulty: 'easy'|'medium'|'hard', tags: [String],
  timeLimitMs: 1000, memoryLimitMb: 256,
  checkerType: 'exact'|'trim'|'float'|'custom', floatEps,
  testcaseSetVersion,             // hash — invalidates the worker's local cache
  source: { site, url, license },  // null for original problems
  isPublic, createdBy, createdAt
}

testcases {
  _id, problemId, idx, isSample,
  inputRef, outputRef,            // S3 keys in V2, inline strings in V1
  bytes
}

submissions {
  _id, userId, problemId, contestId|null,
  language, codeRef,              // S3 key — large code is never inlined
  status: 'QUEUED'|'COMPILING'|'RUNNING'|'AC'|'WA'|'TLE'|'MLE'|'RE'|'CE'|'OLE'|'IE',
  failedTestcaseIdx|null, passedCount, totalCount,
  timeMs, memoryKb, compilerLog, stderrTail,     // truncated to 8 KB
  attempts, judgedBy, submittedAt, judgedAt
}

contests {
  _id, title, startAt, endAt, problems: [{problemId, points}],
  scoring: 'icpc'|'ioi', aiMentorEnabled: false, participants: [userId]
}

ai_interactions {
  _id, userId, problemId, mode, promptTokens, completionTokens,
  cacheHit, leakGuardTriggered, createdAt
}
```

Submission code and testcase data are object-store references, not embedded documents. A 16 MB BSON limit and a contest's worth of source code do not mix.

3.2 Indexes and the query each one serves

An index with no named query is an index nobody can safely delete.

COLLECTION	INDEX	SERVES
`users`	`{username:1}` unique	login, profile page
`users`	`{email:1}` unique	registration conflict check
`problems`	`{slug:1}` unique	`GET /api/problems/:slug`
`problems`	`{isPublic:1, difficulty:1, _id:-1}`	catalogue list + cursor pagination
`problems`	`{tags:1}`	tag filter

COLLECTION	INDEX	SERVES
`testcases`	`{problemId:1, idx:1}` unique	worker fetches the ordered set
`submissions`	`{userId:1, submittedAt:-1}`	"my submissions"
`submissions`	`{problemId:1, userId:1, status:1}`	"have I solved this?" badge
`submissions`	`{contestId:1, submittedAt:1}`	projector rebuild, replay in order
`submissions`	`{status:1, submittedAt:1}` partial: `status ∈ PENDING`	**stuck-job sweeper** — small because it only indexes in-flight rows
`refresh_tokens`	`{tokenHash:1}` unique	refresh rotation
`refresh_tokens`	`{expiresAt:1}` TTL 0	automatic expiry
`outbox`	`{sentAt:1}` partial: `sentAt == null`	relay scan — only unsent rows

3.3 Collections added since the HLD

```
refresh_tokens {
  _id, userId, tokenHash,        // sha256 of the opaque token — never the token itself
  familyId,                      // rotation lineage; reuse revokes the whole family
  replacedBy: ObjectId|null,
  expiresAt, revokedAt, createdAt, userAgent, ip
}

outbox {                         // transactional outbox — see 7.6
  _id, aggregateId,              // submissionId
  topic, key, envelope,          // the exact bytes to publish
  sentAt: Date|null, attempts, createdAt
}

judge_runs {                     // one row per testcase execution, for debugging and the auditor
  _id, submissionId, testcaseIdx,
  cpuMs, wallMs, peakMemKb, exitCode, signal, verdict,
  createdAt                      // TTL 14 days
}
```

3.4 Migrations

Numbered, forward-only, idempotent, run by a job on deploy before the new code starts serving.

```
packages/db/migrations/
  001_initial_indexes.ts
  002_refresh_token_family.ts
  003_outbox.ts
```

Each exports `async up(db)`. Index creation always uses `{ background: true }`. **No migration may rewrite `submissions` in place** — that collection is append-mostly and a full rewrite during a contest is an outage.

04 API Service

4.1 Middleware chain — order is load-bearing

```
requestId            → attaches x-request-id, seeds the OTel span
helmet               → security headers
cors                 → allowlist, credentials: true
bodyParser           → 128 KB limit (source cap is 64 KB, leaves envelope room)
authenticate         → verifies access JWT, populates req.user; never rejects
rateLimit            → keyed by req.user?.id ?? ip, so anonymous abuse is bounded
authorize(role)      → per-route, rejects
validate(zodSchema)  → body/query/params, rejects with 422 + field paths
handler
errorHandler         → last; maps thrown AppError to the envelope in §15
```

`authenticate` never rejects — it only populates. Rejection is `authorize`'s job. Mixing the two is how endpoints accidentally become public.

4.2 Authentication

	ACCESS TOKEN	REFRESH TOKEN
Form	JWT, HS256	256-bit opaque random, base64url
Lifetime	15 min	30 days
Storage	memory in the SPA	httpOnly + Secure + SameSite=Lax cookie, `Path=/api/auth`
Server state	none	`refresh_tokens`, stored as sha256
Payload	`{sub, role, jti, iat, exp}`	—

Rotation with reuse detection. Every refresh issues a new token and sets `replacedBy` on the old one. If a token that already has `replacedBy` is presented, it has been stolen and replayed:

```
POST /api/auth/refresh
  row = find({ tokenHash: sha256(cookie) })
  if !row || row.expiresAt < now       -> 401
  if row.revokedAt                     -> 401
  if row.replacedBy                    -> revoke entire familyId, 401   // reuse detected
  next = issue(userId, familyId = row.familyId)
  row.replacedBy = next._id
  return { accessToken } + Set-Cookie(next)
```

Logout revokes the family. Password change revokes every family for the user.

4.3 Rate limiting

Token bucket in Redis. Must be atomic, so it is a Lua script — a `GET` then `SET` from Node is a race under exactly the load that matters.

```
-- KEYS[1] = bucket key   ARGV = capacity, refillPerSec, nowMs, cost
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or tonumber(ARGV[1])
local ts     = tonumber(b[2]) or tonumber(ARGV[3])
local elapsed = math.max(0, tonumber(ARGV[3]) - ts) / 1000
tokens = math.min(tonumber(ARGV[1]), tokens + elapsed * tonumber(ARGV[2]))
if tokens < tonumber(ARGV[4]) then
  return {0, math.ceil((tonumber(ARGV[4]) - tokens) / tonumber(ARGV[2]))}
end
redis.call('HMSET', KEYS[1], 'tokens', tokens - tonumber(ARGV[4]), 'ts', ARGV[3])
redis.call('EXPIRE', KEYS[1], 3600)
return {1, 0}
```

ROUTE	CAPACITY	REFILL	COST
`POST /api/run`	10	1 / 6 s	1
`POST /api/submissions`	20	1 / 3 s	1
`POST /api/ai/ask`	6	1 / 30 s	1
`POST /api/auth/login`	5 per IP	1 / 60 s	1
everything else	300	5 / s	1

If Redis is unreachable the limiter fails open and logs. A cache outage must not take down login. The queue is the real backpressure for `/submit`, not the limiter.

4.4 Pagination

Cursor-based, never `skip`. `skip` on page 400 of a contest's submissions scans 400 pages.

```
GET /api/problems?limit=25&cursor=<base64 of last _id>
  find({ isPublic: true, ...filters, _id: { $lt: decoded } }).sort({_id:-1}).limit(limit+1)
  → { items: first(limit), nextCursor: items.length > limit ? encode(last._id) : null }
```

4.5 The submit path

```
POST /api/submissions
  1. validate(SubmitBody)
  2. idempotency: key = req.header('Idempotency-Key')
     if key: SET idem:{userId}:{key} <placeholder> NX EX 600
             if not acquired -> return the stored submissionId (200, not 202)
  3. problem = problems.findOne({_id, isPublic:true})            -> 404
  4. if contestId: assert now within [startAt, endAt] and user registered  -> 409
  5. codeRef = s3.put(`code/${submissionId}.${ext}`, code)       // before the DB row
  6. submissions.insertOne({ _id: submissionId, status:'QUEUED', attempts:0, ... })
  7. outbox.insertOne({ topic:'submission.created', ... })
  8. queue.add('judge', { submissionId }, { priority: contestId ? 1 : 5,
                                       attempts: 3, backoff:{type:'exponential', delay:2000} })
  9. redis.SET idem:{...} submissionId EX 600
 10. return 202 { submissionId, status:'QUEUED' }
```

Step 5 before step 6 is deliberate. An orphaned S3 object is garbage; a submission row pointing at a missing object is a permanent `IE`. Order operations so the survivable failure is the one that happens.

05 Queue Layer

```
// packages/queue/src/judge.ts
export const judgeQueue = new Queue<JudgeJob>('judge', { connection });
export type JudgeJob = { submissionId: string };   // reference only, never the code
```

SETTING	VALUE	WHY
`priority`	1 contest · 3 `/run` · 5 practice	Interactive and contest traffic jump a practice backlog
`attempts`	3	Then `IE` and DLQ
`backoff`	exponential, 2 s base	A transient Docker hiccup resolves; a poison pill does not
`lockDuration`	90 s	Longer than the worst 25 s judgement, with headroom
`lockRenewTime`	30 s	Worker renews while genuinely alive
`removeOnComplete`	1000	Recent history for debugging
`removeOnFail`	false	Failures are evidence

A separate `judge-interactive` queue for `/run` is created in V2 when a single queue's priority handling stops being enough. Until then, priority on one queue is sufficient and simpler.

06 Sandbox Module

The only module allowed to spawn a process. Everything about it is a contract.

```
export interface SandboxRequest {
  image:   string;
  argv:    string[];
  stdin:   Buffer;
  cpuset:  string;                  // "3" — the pinned core for this slot
  limits: {
    cpuMs: number; wallMs: number;  // wallMs is the kill timer, cpuMs decides TLE
    memoryMb: number; pids: number;
    stdoutBytes: number; fileSizeBytes: number;
  };
}

export interface SandboxResult {
  outcome: 'exited' | 'timeout' | 'oom' | 'spawn_error';
  exitCode: number | null;
  signal:   string | null;          // 'SIGSEGV' | 'SIGKILL' | ...
  cpuMs:    number;                 // from the cgroup, see §17
  wallMs:   number;
  peakMemKb: number;
  stdout: Buffer; stderr: Buffer;
  stdoutTruncated: boolean;
}

export function runInSandbox(req: SandboxRequest): Promise<SandboxResult>;
```

6.1 Lifecycle

```
create   docker create <flags> <image> <argv>      -> containerId
start    docker start -ai <containerId>            -> attach stdio
feed     write stdin, then end it                  // a program blocked on stdin must see EOF
watch    wallMs timer      -> docker kill -s KILL
         wallMs + 2000     -> docker rm -f, outcome = 'timeout'
collect  docker inspect    -> ExitCode, OOMKilled, State.StartedAt/FinishedAt
metrics  read cgroup files BEFORE removal          // see §17 — this is why --rm is gone
remove   docker rm -f                              // in finally, always
```

6.2 Output handling

stdout is consumed as a stream with a running byte count. At `stdoutBytes` the stream is destroyed, `stdoutTruncated` is set, and the container is killed — buffering an unbounded pipe is how a worker OOMs on the *host*.

stderr is capped at 8 KB, keeping the **last** 8 KB rather than the first: the useful part of a stack trace is at the end.

6.3 Outcome to verdict

Applied in this order. The first match wins.

CONDITION	VERDICT
`outcome == 'spawn_error'`	`IE` — our fault, retry the job
`outcome == 'oom'` or `inspect.OOMKilled`	`MLE`
`outcome == 'timeout'` or `cpuMs > limits.cpuMs`	`TLE`

CONDITION	VERDICT
`stdoutTruncated`	`OLE`
`signal != null`	`RE` — surface the signal name to the user
`exitCode != 0`	`RE`
output mismatch	`WA`
otherwise	`AC`

`MLE` before `TLE` matters: a program thrashing against its memory limit usually trips the wall timer too, and reporting `TLE` sends the user optimising the wrong axis.

6.4 Container flags

Moved here from HLD §10.6, which now states the guarantees rather than the CLI. Every flag exists because of a specific attack.

```
docker create -i \
  --network=none \                    no exfiltration, no mining, no callbacks
  --memory=256m --memory-swap=256m \  swap must equal memory, or the limit is meaningless
  --cpus=1 --cpuset-cpus="<slot>" \   a dedicated core => reproducible timings
  --pids-limit=64 \                   the fork bomb dies here
  --ulimit fsize=67108864 \           64 MB write cap — no filling the disk
  --ulimit nofile=64 \
  --read-only \                       rootfs is immutable
  --tmpfs /box:rw,noexec,nosuid,size=64m \   only writable dir, cannot exec what it writes
  --cap-drop=ALL \
  --security-opt=no-new-privileges \
  --security-opt seccomp=./judge-seccomp.json \
  --user 65534:65534 \                nobody
  oj/run-cpp:1 /box/main
```

Note the absence of `--rm` and of `--userns=host`. The cgroup must outlive the process (§17.1), and user-namespace remapping is left at the daemon default so container root maps to an unprivileged host uid.

Images are pinned by digest and rebuilt on a schedule. `latest` is banned — a runner image that changes underneath the compile cache produces binaries the cache key does not describe.

6.5 Language registry

Adding a language is a config entry plus a Dockerfile, never a code change. The MVP ships C++, Python and Java — the minimum of three — then C and JavaScript.

```
const LANGUAGES = {
  cpp17:   { image:'oj/run-cpp:1',  file:'main.cpp',
             compile:'g++ -O2 -std=c++17 main.cpp -o main',  run:'./main' },
  c:       { image:'oj/run-cpp:1',  file:'main.c',
             compile:'gcc -O2 main.c -o main',               run:'./main' },
  python3: { image:'oj/run-py:1',   file:'main.py',
             compile:null,                                   run:'python3 main.py' },
  java17:  { image:'oj/run-java:1', file:'Main.java',
             compile:'javac Main.java',
             run:'java -Xss64m -XX:+UseSerialGC Main',
             note:'public class MUST be Main — filename is forced, a mismatch is a CE' },
  node20:  { image:'oj/run-node:1', file:'main.js',
             compile:null,                                   run:'node main.js' },
};
```

`compileTimeoutMs` is 10,000 for all. Compiled artefacts are cached by `sha256(code + language + imageDigest)`.

6.6 The host executor is development-only

V0 ships a `HostExecutor` that shells out with `child_process` and no isolation at all. It exists so the editor and the `/run` path can be built before the sandbox is ready. **It must never run in a deployed environment**, and four independent guards enforce that:

#	GUARD	MECHANISM
1	Boot	The module throws at import if `NODE_ENV === 'production'` or `ALLOW_HOST_EXECUTOR !== 'true'`. Fails at startup, not on the first request.
2	Build	CI greps the production bundle for any import of `packages/executor-host`; a match fails the build.
3	Config	`ALLOW_HOST_EXECUTOR` is absent from every non-local env file, and the zod env schema rejects it being `true` while `NODE_ENV` is `production`.
4	Deployment	V0 is never deployed. Staging begins at V1, after the sandbox lands.

The package is deleted outright once V1 merges. A dev-only escape hatch that survives into the repo's third month becomes production code by accident.

07 Judge Worker

7.1 Structure

```
JudgeWorker
├── SlotManager        N slots, slot i pinned to cpuset core i
├── ArtifactStore      S3 get code
├── TestcaseCache      local disk, LRU by bytes, key problemId:version
├── CompileCache       local disk, LRU, key sha256(code + language + imageDigest)
├── SandboxRunner      §6
├── Comparators        exact | trim | float | custom
└── VerdictWriter      CAS to Mongo + outbox insert, in one operation
```

`N = availableCores - 2`. Two cores are reserved for the agent, the Docker daemon and the worker's own event loop. A worker that saturates every core produces unreliable timings for every submission on the box, including the ones it is not judging.

7.2 Caches

CACHE	KEY	EVICTION	INVALIDATION
Testcases	`{problemId}:{testcaseSetVersion}`	LRU, 8 GB budget	Version bump makes a new key; the old one ages out
Compiled binary	`sha256(code + language + imageDigest)`	LRU, 4 GB	Image digest is in the key, so a runner rebuild invalidates naturally

Including the image digest in the compile key is what stops a rebuilt `g++` image from silently serving binaries built by the previous compiler.

7.3 Comparators

The single largest source of "the judge is broken" reports. Specified exactly.

```
trim(actual, expected):          # the default
  1. decode UTF-8, replace \r\n and \r with \n
  2. split into lines
  3. rstrip each line of spaces and tabs
  4. drop trailing empty lines from both sides
  5. equal iff the line arrays are equal

exact(actual, expected):
  byte-for-byte equality, no normalization at all

float(actual, expected, eps):
  1. tokenize both on any whitespace run
  2. token counts must match, else WA
  3. per pair (a, b):
       if both parse as finite floats:
          ok iff |a-b| <= eps  OR  |a-b| / max(1, |b|) <= eps
       else:
          ok iff a == b as strings
```

EDGE CASE	BEHAVIOUR
Output ends without a trailing newline	`trim` accepts, `exact` rejects
Windows `\r\n` from a user's file	`trim` accepts
`1.0` vs `1`	`float` accepts, `trim` rejects
`-0.0` vs `0.0`	`float` accepts (difference is 0)
`nan`, `inf`	Never accepted — not finite, falls to string compare
Trailing spaces on a line	`trim` accepts
Extra blank line at EOF	`trim` accepts

EDGE CASE	BEHAVIOUR
Blank line in the *middle*	Rejected by all — it is a real structural difference

Every row above is a unit test. They ship before the comparator does.

7.4 Main loop

```
onJob({ submissionId }):
  slot = slots.acquire()                                  # blocks until one frees
  try:
    sub = CAS(submissionId, PENDING -> 'COMPILING')
    if !sub: return                                       # another worker owns it

    problem   = problems.get(sub.problemId)
    code      = artifacts.getCode(sub.codeRef)
    testcases = testcaseCache.get(problem._id, problem.testcaseSetVersion)

    bin = compileCache.get(key) ?? compile(code, lang, slot)
    if bin.failed: return finish('CE', { compilerLog: bin.stderr })

    CAS(submissionId, 'COMPILING' -> 'RUNNING')

    for tc in testcases:                                  # sequential, see HLD §9.5
      r = runInSandbox({ ..., stdin: tc.input, cpuset: slot.core })
      record(judge_runs, r)
      v = mapOutcome(r) ?? compare(r.stdout, tc.expected, problem.checkerType)
      if v != 'AC': return finish(v, { failedTestcaseIdx: tc.idx, ...stats })

    return finish('AC', { timeMs: max(cpuMs), memoryKb: max(peakMemKb) })
  finally:
    slot.release(); rmrf(scratch)
```

`finish()` is the only path that writes a terminal status, and it always goes through §7.6.

7.5 The stuck-job sweeper

BullMQ redelivers on lock expiry, but a submission can still be stranded — for example if the row was created and the enqueue then failed. A cron every 60 s:

```
submissions.find({ status: { $in: PENDING }, submittedAt: { $lt: now - 5min } })
  → attempts < 3 ? re-enqueue : finish('IE')
```

This is what the partial index in §3.2 exists for.

7.6 Transactional outbox

The bug this prevents: the worker writes `AC` to Mongo, then crashes before producing to Kafka. The submission looks solved, the leaderboard never learns, and the contestant's rank is silently wrong.

The verdict write and the event are one atomic operation:

```
const session = client.startSession();
await session.withTransaction(async () => {
```

```
  const r = await submissions.updateOne(
    { _id, status: { $in: PENDING } },
    { $set: { status, judgedAt: new Date(), ...stats } },
    { session });
  if (r.matchedCount === 0) throw new AlreadyFinished();   // aborts, emits nothing
  await outbox.insertOne({ topic:'submission.judged', key: contestId ?? userId,
                           envelope, sentAt: null }, { session });
});
```

A relay loop polls `outbox.find({ sentAt: null })`, produces to Kafka, and sets `sentAt`. Duplicates are possible — publish then crash before marking — which is why every consumer is idempotent on `eventId` (§9.2). At-least-once delivery with idempotent consumers, not exactly-once, which does not exist.

08 Event Layer

8.1 Producer

```
new Kafka({ clientId, brokers }).producer({
  idempotent: true,              // dedup on broker retry
  maxInFlightRequests: 5,
  transactionTimeout: 30000,
});
send({ topic, messages: [{ key, value: JSON.stringify(envelope) }], acks: -1 });
```

`acks: -1` (all in-sync replicas) is not negotiable for `submission.judged`. It is the record of a contestant's result.

8.2 Consumer

SETTING	VALUE	WHY
`groupId`	one per consumer role	`leaderboard-projector`, `analytics`
`sessionTimeout`	30 s	Long enough to survive a GC pause
`heartbeatInterval`	3 s	
`autoCommit`	**false**	Commit only after the projection is durable, or a rebalance loses an update
`maxBytesPerPartition`	1 MB	Envelopes are small; a big fetch just adds latency

09 Leaderboard Projector

9.1 Scoring

ICPC rules, stated unambiguously because "penalty" means three different things across judges.

```
per (contest, user, problem):
  solved        = any AC with submittedAt <= contest.endAt
  solveMinute   = floor((firstAC.submittedAt - contest.startAt) / 60000)
  wrongBefore   = count of submissions before firstAC with status in
                  {WA, TLE, MLE, RE, OLE}          # CE and IE do NOT count
  problemPenalty = solved ? solveMinute + 20 * wrongBefore : 0

per (contest, user):
  solvedCount = sum(solved)
  penalty     = sum(problemPenalty)
  zscore      = solvedCount * 1e9 - penalty
```

`CE` not counting is a deliberate rule choice, matching Codeforces. It is written here so nobody re-derives it differently in six weeks.

9.2 Idempotent projection

```
on(envelope):
  if SADD seen:{contestId} envelope.eventId == 0: return    # already applied
  EXPIRE seen:{contestId} 172800                            # 48 h, outlives any contest

  state = HGETALL contest:{c}:user:{u}                      # per-problem state
  apply the verdict to state[problemId]
  if state unchanged: return                                # WA after AC changes nothing
  HSET  contest:{c}:user:{u} ...
  ZADD  contest:{c}:board {zscore} {userId}
  PUBLISH contest:{c}:delta { userId, rank, solvedCount, penalty }
  commitOffset()
```

Applying a verdict to a user's state is a pure function of the state and the event, so replaying the topic from offset 0 rebuilds the identical board. That is the recovery procedure — there is no backup to restore.

9.3 Reads

```
GET /api/contests/:id/leaderboard?limit=50&offset=0
  ZREVRANGE contest:{id}:board offset offset+limit-1 WITHSCORES
  → unpack: solvedCount = floor(score / 1e9), penalty = solvedCount*1e9 - score
  → hydrate usernames from a Redis hash, not Mongo
```

`ZREVRANK` gives one user their own rank in O(log N) without reading the page they are not on.

10 Realtime

10.1 SSE — one submission

```
GET /api/submissions/:id/stream        Accept: text/event-stream
  authorize: req.user.id === submission.userId, else 403
  if already terminal: send one event, close                # no dangling connection
  subscribe redis: sub:{id}
  every 15 s: write ':ping\n\n'                             # defeats idle proxy timeouts
```

```
  on terminal event: write, then close
  hard cap 120 s: close with `event: timeout`, client falls back to polling
```

The worker `PUBLISH`es to `sub:{id}` on every status transition. Redis pub/sub is fire-and-forget, which is correct here — the durable record is Mongo, and a client that missed a message re-reads `GET /api/submissions/:id` on reconnect.

10.2 WebSocket — contest standings

One room per contest. On join, send a full top-50 snapshot with a `seq`; then send deltas carrying `seq+1`. A client that sees a gap in `seq` requests a fresh snapshot. Without the sequence number a dropped delta leaves a permanently wrong board on that client, and nobody notices until someone screenshots it.

11 AI Mentor

11.1 Mode registry

```
interface Mode {
  id: 1|2|3|4|5|6;
  needsExecution: boolean;
  model: 'fast' | 'strong';
  maxOutputTokens: number;
  contextParts: Array<'statement'|'constraints'|'userCode'|'recentVerdicts'
                      |'failedSampleIO'|'trace'|'timingCurve'|'catalogueByTag'>;
  systemPrompt: string;
}
```

MODE	EXECUTION	MODEL	CONTEXT PARTS
1 Intuition Engine	no	fast	statement, constraints
2 Brainstorming Partner	no	strong	statement, userCode, recentVerdicts
3 Dry-Run Simulator	**yes**	strong	statement, userCode, trace, failedSampleIO
4 Complexity Auditor	**yes**	strong	statement, constraints, userCode, timingCurve
5 Conceptual Translator	no	fast	statement
6 Foundations Architect	no	fast	statement, catalogueByTag

11.2 Context builder

Assembled server-side. The client's `editorBuffer` is the only thing it contributes, and it is capped at 64 KB.

```
build(userId, problemId, mode):
  problem = problems.get(problemId)
  if problem.contestId is live and !contest.aiMentorEnabled: throw AI_DISABLED_IN_CONTEST
  parts = {}
  if 'statement'      in mode.contextParts: parts.statement = problem.statement
  if 'recentVerdicts' in mode.contextParts:
      parts.recentVerdicts = submissions.find({userId, problemId})
        .sort({submittedAt:-1}).limit(5)
        .project({ status:1, failedTestcaseIdx:1, timeMs:1 })     # never other users' data
  if 'failedSampleIO' in mode.contextParts:
      tc = testcases.findOne({ problemId, idx: lastFailedIdx })
      parts.failedIO = tc.isSample ? tc : null                    # hidden cases stay hidden
  ...
```

The `isSample` guard is the important line. Leaking a hidden test case through the mentor is the same failure as leaking it through the API, and it is much easier to do by accident.

11.3 Mode 3 — the tracer

LANGUAGE	MECHANISM	SHIPS
`python3`	`sys.settrace` line hook, records `(lineno, changed locals)`	V3
`node20`	`node --inspect-brk` + CDP `Debugger.stepOver`	V3
`cpp17`, `c`	`gdb --batch` with a step script, `-g -O0` build	V3.1
`java17`	JDI stepping	not planned — falls back to mode 2

The trace runs in the same sandbox with a hard cap of **2,000 events** and a 3 s wall limit. Truncated traces are labelled as such in the prompt so the model does not reason about a run it cannot see the end of.

```
{"line": 7, "delta": {"i": 3, "total": 12}}
{"line": 8, "delta": {"total": 19}}
```

If a language has no tracer, the mode is not offered for it. Falling back to an LLM pretending to trace, under a UI that says "real execution", is worse than not shipping the feature.

11.4 Mode 4 — the complexity fitter

```
sizes = [1e2, 1e3, 1e4, 1e5]
for n in sizes:
    input = problem.generator(n)                 # setter-provided, else a typed default
    t[n] = median(3 runs).cpuMs
drop points where t < 5ms                        # below the measurement noise floor
if fewer than 3 points remain: return INCONCLUSIVE

# power law: log t = a·log n + b
a, r2_pow = linreg(log(sizes), log(t))
# exponential: log t = c·n + d
c, r2_exp = linreg(sizes, log(t))

if r2_exp > r2_pow and r2_exp > 0.95: return 'exponential'
if r2_pow < 0.90: return INCONCLUSIVE
classify a: <0.35 O(1)/O(log n) · 0.8–1.25 O(n) · 1.25–1.6 O(n log n)
            1.7–2.3 O(n²) · 2.7–3.3 O(n³) · else 'O(n^{a:.1f})', unusual'
```

Returning `INCONCLUSIVE` honestly is a feature. A confident wrong complexity sends someone rewriting a correct solution.

11.5 Leak guard

Runs on the complete response before a single token reaches the client, which is why mode 3 and 4 responses are buffered rather than streamed. Modes 1, 2, 5 and 6 stream, with the guard applied per completed code block.

```
block if ANY:
  A. a fenced code block with >= 8 non-comment lines
     AND contains a loop or recursion
     AND contains a return/print of a result
     AND the fence language matches the submission language
  B. normalized similarity to any editorial solution > 0.75
     (strip comments and whitespace, rename identifiers to positional tokens,
      then token-level Jaccard over 4-grams)
  C. the response contains > 60% of the identifiers present in an editorial

on block: replace with the mode's canned nudge, increment leak_guard_triggered,
          log { userId, problemId, mode, rule } — never log the blocked text
```

Rule A alone would block a legitimate 10-line explanation of a helper. That is the intended trade: **false positives are cheap, a leaked solution is not.**

11.6 Budget and cache

```
cacheKey = sha256(problemId + mode + normalize(code))   # normalize: strip comments + whitespace
budget:  INCRBY ai:tokens:{userId}:{yyyy-mm-dd} <tokens>, EXPIRE 172800
         reject with AI_BUDGET_EXCEEDED above the daily cap
```

The budget is checked *before* the call using the prompt token count plus `maxOutputTokens` as the pessimistic estimate, then reconciled to actual usage after. Checking only after means one expensive request can blow through any cap.

11.7 Independence from the judge

The mentor is an enhancement. The judge is the product. The judging path must never acquire a dependency on the mentor, and that is enforced rather than assumed.

RULE	ENFORCEMENT
`api` and `judge-worker` never import `ai-mentor`	Lint rule plus a CI dependency-graph check that fails the build
No synchronous call from the request or judge path into the mentor	The mentor is reachable only at `POST /api/ai/*`; nothing internal calls it
The mentor holds a read-only database credential	Separate Mongo user with no write grant on any collection
The mentor never writes `submissions`, `contests` or standings	Follows from the credential
Mentor down means 503 on `/api/ai/*` and nothing else	Separate deployment, separate health check, its own autoscaler
Global kill switch	`AI_ENABLED=false` returns 503 from `/api/ai/*` immediately; no other behaviour changes
Provider timeouts cannot consume API capacity	The mentor owns its own process pool; a hung LLM call blocks a mentor worker, never an API worker

Verification, not assertion. The V4 chaos drill scales `ai-mentor` to zero replicas in the middle of a mock contest. Pass condition: every submission is still judged, standings keep updating, and the only user-visible change is the mentor panel reporting itself unavailable.

12 Ingestor

```
interface SourceAdapter {
  id: string;                                   // 'codeforces'
  fetchIndex(since?: Date): AsyncIterable<ProblemRef>;
  fetchOne(ref: ProblemRef): Promise<RawProblem>;
  readonly policy: 'metadata-only' | 'full';    // see HLD §8.7
}
```

The normalizer maps `RawProblem` to our schema, generates a slug, and upserts on `(source.site, source.externalId)` so a re-run updates rather than duplicates. `policy: 'metadata-only'` adapters have their `statement` field dropped at the normalizer, not at the adapter — one enforcement point, not one per source.

12.1 Testcase generation

```
generator.py <n> <seed>   -> stdout: one test input
brute.<lang>              -> reads that input, writes the reference output
```

Both run in the standard sandbox with a 10 s limit. The pipeline generates M cases, deduplicates by input hash, and stores the set with `testcaseSetVersion = sha256(sorted input hashes)`. Requiring the brute force to run in the same sandbox means a generator that hangs fails the import instead of the platform.

13 Frontend

ROUTE	DATA	NOTES
`/`	`GET /api/problems`	Cursor pagination, filters in the URL query so a filtered list is shareable
`/problems/:slug`	problem + samples	Monaco, language selector, Run and Submit
`/submissions/:id`	SSE stream	Falls back to 2 s polling if SSE closes
`/contests/:id`	WS room	Snapshot then deltas, §10.2
`/u/:username`	profile	

`useSubmission(id)` is the one hook that matters: it opens SSE, reduces status transitions into local state, closes on terminal, and falls back to polling on error. Every other component reads its result. Submission state lives in exactly one place.

Editor buffer is persisted to `localStorage` per `(problemId, language)` and restored on mount. Losing forty minutes of work to an accidental refresh is the single worst experience this product can deliver, and it costs four lines to prevent.

14 Configuration

VARIABLE	SERVICE	DEFAULT	NOTES
`MONGO_URI`	api, worker, projector	—	
`REDIS_URL`	all	—	
`KAFKA_BROKERS`	api, worker, projector	—	V2+
`S3_ENDPOINT` `S3_BUCKET`	api, worker	MinIO in dev	
`JWT_SECRET`	api	—	Rotating it invalidates every access token, which is the point
`JUDGE_SLOTS`	worker	`cores - 2`	
`JUDGE_CPUSET_BASE`	worker	`2`	First core the slots may use
`SANDBOX_IMAGE_TAG`	worker	pinned digest	Never `latest`

VARIABLE	SERVICE	DEFAULT	NOTES
`AI_PROVIDER` `AI_API_KEY`	ai-mentor	—	
`AI_DAILY_TOKEN_CAP`	ai-mentor	`150000`	Per user
`LOG_LEVEL`	all	`info`	

Every service validates its own env with zod at boot and **exits non-zero on a missing variable**. A service that starts with a missing config and fails on the first request fails in production instead of in the deploy.

15 Error Envelope and Codes

```
{ "error": { "code": "RATE_LIMITED", "message": "Too many submissions.",
             "details": { "retryAfterSec": 4 }, "requestId": "01J..." } }
```

CODE	HTTP	MEANING
`VALIDATION_FAILED`	422	`details.fields` carries zod paths
`UNAUTHENTICATED`	401	Missing or expired access token
`TOKEN_REUSE_DETECTED`	401	Family revoked, re-login required
`FORBIDDEN`	403	Authenticated, wrong role or not the owner
`NOT_FOUND`	404	
`CONTEST_NOT_RUNNING`	409	Submit outside the window
`RATE_LIMITED`	429	`Retry-After` header set
`AI_DISABLED_IN_CONTEST`	409	
`AI_BUDGET_EXCEEDED`	429	Resets at UTC midnight
`INTERNAL`	500	Message is always generic; detail goes to logs only

`requestId` appears in the response, in every log line, and as the OTel trace id. A user reporting a bug can paste one string that finds everything.

16 Concurrency and Failure Edge Cases

The section to re-read before saying a module is done.

SCENARIO	HANDLING
Two workers lease the same job after a lock expiry	CAS on status; the loser's `matchedCount` is 0 and it exits silently
Worker dies after the DB write, before the Kafka emit	Impossible — same transaction, §7.6. The relay republishes
Relay publishes, then dies before marking `sentAt`	Duplicate event; consumer dedups on `eventId`, §9.2
Submission row created, enqueue then fails	Sweeper re-enqueues after 5 min, §7.5
Contest ends while a submission is being judged	Counted if `submittedAt <= endAt`. Judging time is ours, not the contestant's
Test cases edited during a live contest	`testcaseSetVersion` bumps; in-flight judgements complete on the old set and are automatically re-queued for rejudge at low priority
Same code submitted twice	Compile cache hits, but it is judged again — the user may be checking timing variance
Rejudge of a whole problem	Enqueue every submission at priority 9; contest boards rebuild from the topic afterwards
Redis is flushed mid-contest	Projector replays `submission.judged` from the contest-start offset, §9.2
Redis is down at login	Rate limiter fails open; login still works, §4.3
A worker's clock is wrong	Never consulted. Timings come from the cgroup, ordering from Kafka offsets
User deletes their account mid-contest	Soft delete. Submissions and standings are contest records and are retained, anonymized
Docker daemon dies on a worker	Health check fails, the node drains, in-flight jobs redeliver elsewhere

17 Corrections to the HLD

Detailed design surfaced two places where the HLD was wrong. **Both have since been applied to the HLD**, which now states sandbox guarantees rather than flags; the flag list in §6.4 is authoritative.

17.1 `--rm` and cgroup metrics are mutually exclusive

HLD §10.6 shows `docker run --rm` and HLD §9.5 says CPU time is read from the cgroup after exit. Both cannot be true: `--rm` deletes the container and its cgroup the instant it exits, so there is nothing left to read.

Resolution. Drop `--rm`. Use the explicit lifecycle in §6.1 — `create`, `start`, `wait`, read `cpu.stat` and `memory.peak`, then `docker rm -f` in a `finally`. Removal is the worker's responsibility, not Docker's.

```
/sys/fs/cgroup/system.slice/docker-<containerId>.scope/cpu.stat      -> usage_usec
/sys/fs/cgroup/system.slice/docker-<containerId>.scope/memory.peak   -> bytes
```

Requires cgroup v2, which is the default on every current Linux distribution.

17.2 Timing on macOS development machines is approximate

Docker Desktop runs containers inside a VM, so those cgroup paths are not reachable from the host. On macOS the sandbox falls back to wall-clock timing and the `OOMKilled` flag from `docker inspect`.

Consequence: `TLE` and `MLE` verdicts are indicative, not authoritative, in local development. The golden-verdict suite (HLD §13) must therefore run in Linux CI, not on a developer's laptop, or timing-sensitive cases will pass locally and fail in CI — or worse, the reverse.

18 Build Order

Each row is done when its tests pass and the next row can depend on it.

#	MODULE	DONE WHEN
1	`contracts`	Types compile, zod schemas round-trip
2	`db` + migrations	Indexes exist, `explain()` confirms each query in §3.1 uses one
3	`sandbox`	Fork bomb → `RE`, infinite loop → `TLE`, 1 GB alloc → `MLE`, network call fails
4	Comparators	Every row of the §7.3 edge case table is a passing test
5	`api` auth + problems	Reuse detection revokes a family; rate limiter is atomic under 100 parallel requests
6	`queue` + `judge-worker`	A submission goes `QUEUED → AC` end to end
7	Outbox + relay	Kill the worker between write and publish; the event still arrives
8	`events` + `projector`	Replay from offset 0 reproduces a board byte for byte
9	SSE + WebSocket	Standings survive a dropped delta via the `seq` gap check
10	`ai-mentor` modes 1, 2, 5, 6	Leak guard blocks a pasted full solution
11	Modes 3 and 4	Tracer output is real; fitter returns `INCONCLUSIVE` on noise
12	`ingestor`	Re-running an import updates rather than duplicates

Rows 3 and 4 before row 6 is the important ordering. A judge built on an unverified sandbox produces verdicts nobody can trust, and every bug after that looks like a judge bug.